

DevFlow — Minimal Development Guideline

Goal: Build a small real-time team collaboration platform to learn and practice Socket.IO, Redis, MongoDB, and modern full-stack development.

1. Project Goal

- User Authentication
- Workspace & Projects
- Task Management
- Team Members
- Real-Time Chat
- Online / Offline Status
- Real-Time Notifications

2. Tech Stack

Layer	Technology
Frontend	Next.js, React, TypeScript, Tailwind CSS, Socket.IO Client
Backend	Node.js, Express.js, TypeScript, Socket.IO, JWT
Database	MongoDB + Mongoose
Real-Time / Cache	Redis

3. Basic Architecture

Next.js / React → Node.js + Express → Socket.IO → Redis → MongoDB

- **MongoDB:** Permanent application data.
- **Redis:** Presence, cache, and Pub/Sub.
- **Socket.IO:** Real-time communication.

4. Development Steps

Step 1 — Project Setup — Create frontend and backend folders, initialize Git, and configure environment variables.

Step 2 — Backend — Set up Express, TypeScript, MongoDB/Mongoose, JWT, Socket.IO, and Redis. Add GET /api/health.

Step 3 — Authentication — Implement Register, Login, Logout, and Get Current User using JWT.

Step 4 — Workspace & Project — Create User, Workspace, Project, and Task models with CRUD APIs.

Step 5 — Frontend — Create /login, /register, /dashboard, and /projects/[id]. Show projects, tasks, members, and notifications.

Step 6 — Socket.IO — Connect frontend and backend. Handle connection/disconnection and use rooms such as project:{projectId}.

Step 7 — Real-Time Chat — Implement message:send and message:new. Save messages in MongoDB and deliver them through Socket.IO.

Step 8 — Typing & Presence — Implement typing:start, typing:stop, user:online, and user:offline. Use Redis for online users.

Step 9 — Real-Time Task Updates — Update MongoDB, emit a Socket.IO event, and update project members without refreshing the page.

Step 10 — Notifications — Notify users when tasks are assigned/updated or when new messages arrive. Store notifications in MongoDB.

Step 11 — Redis — Use Redis for presence, basic caching, and Socket.IO Pub/Sub. Add the Socket.IO Redis Adapter when scaling to multiple servers.

Step 12 — Testing — Test authentication, task CRUD, chat, typing, online/offline status, notifications, and reconnect behavior.

Step 13 — Deployment — Deploy with Docker, Nginx, MongoDB, Redis, Node.js, and Next.js.

5. Production Architecture

User → Nginx → Next.js → Node.js + Socket.IO → Redis / MongoDB

6. MVP Features

Authentication • Workspace • Project • Task Management • Team Members • Real-Time Chat • Typing Indicator • Online/Offline Status • Real-Time Notifications • Redis • MongoDB • Socket.IO

7. Development Principle

First build the normal CRUD application → then add Socket.IO → then add Redis → finally deploy.

Keep the MVP focused. Avoid file uploads, advanced analytics, email systems, and complex permissions at the beginning.